

The Harmonic Integer Processor: An AI Chip Without Floating-Point Units

Replacing IEEE 754 Float Logic with i64 Multiply-Accumulate Arrays, Barrel Shifters, and VFR Shell Instructions for Exact Deterministic AI Computation

Registry: [[CKS-MATH-136-2026](#)]

Series Path: [[CKS-0-2026](#)] → [[CKS-MATH-0-2026](#)] → [[CKS-MATH-1-2026](#)] → [[CKS-MATH-10-2026](#)] → [[CKS-MATH-104-2026](#)] → [[CKS-MATH-135-2026](#)] → [[CKS-MATH-136-2026](#)]

Parent Framework: [[CKS-0-2026](#)]

DOI: 10.5281/zenodo.18959994

Date: March 11, 2026

Domain: Computer Architecture / Hardware Design / AI Accelerators

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [[CKS-NEXT-1-2026](#)]

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [[CKS-LEX-12-2026](#)]

ABSTRACT

Current AI accelerators dedicate 55-65% of their die area to floating-point and tensor core logic implementing IEEE 754 arithmetic — a standard designed in 1985 for scientific computing, not for neural network training. This paper specifies a processor architecture that removes all floating-point hardware and replaces it with integer-only computation units optimized for VFR shell arithmetic ([[CKS-MATH-134-2026](#)]). The Harmonic Integer Processor (HIP) contains: i64 multiply-accumulate arrays for matrix operations,

barrel shifters for harmonic octave scaling (all divisions are bit shifts by multiples of 5), i128 accumulators for overflow-safe intermediate results, fused multiply-accumulate-shift (FMAS) instruction units, R-zero bitmap units for convergence detection, shared octave registers eliminating per-element scale storage, VFR shell transition units for weight updates, hardware GCD units for VFR simplification, and lattice address units for structured memory access patterns. We project: (1) **4-5 $\times$ integer throughput** compared to current GPU integer capability by filling die area freed from float units with integer MACs, (2) **~75% power consumption** because integer multiply-accumulate is simpler than IEEE 754 float multiply with rounding, normalization, denormal handling, and exception logic, (3) **Perfect determinism** because integer arithmetic is associative and commutative regardless of reduction order, thread scheduling, or hardware implementation, (4) **Zero division cycles** in the computation hot path because all VFR scale conversions are bit shifts, (5) **No NVIDIA dependency** because the chip requires no tensor core IP, no float pipeline IP, and no proprietary mixed-precision logic, (6) **Fabricable under export restrictions** because the design is simpler than current GPUs, with lower transistor counts per functional unit and no requirement for advanced float logic, (7) **Complete AI training capability** as demonstrated by [CKS-MATH-134-2026] showing neural network training with zero floating-point operations. The design is specified at the functional unit level. Physical layout, process node selection, and fabrication are outside the scope of this paper.

Central claim: The majority of transistors on current AI accelerators implement arithmetic that is not required for neural network training. Removing those transistors and replacing them with integer units produces a simpler, faster, more power-efficient, perfectly deterministic AI processor that any semiconductor fabricator can manufacture.

I. WHAT CURRENT AI CHIPS SPEND THEIR SILICON ON

1.1 The H100 Die

The NVIDIA H100 GPU, the current standard for AI training, contains approximately 80 billion transistors on an 814 mm² die fabricated at TSMC 4N process. The die area is allocated approximately as follows:

Functional Block	Approximate Die Area	Purpose
Tensor Cores (FP)	25-30%	BF16/FP16/FP8 matrix multiply
CUDA Cores (FP)	15-20%	General float arithmetic
CUDA Cores (INT)	5-8%	Integer arithmetic (secondary)
Register Files	10-15%	Operand storage
Shared Memory / L1	8-12%	On-chip data storage
L2 Cache	8-10%	Second-level cache
Memory Controllers	5-8%	HBM3 interface
Interconnect	5-8%	NVLink, PCIe
Other	5-10%	Schedulers, decoders, misc

Approximately 55-65% of the die is dedicated to floating-point computation: tensor cores and CUDA float units. These implement IEEE 754 arithmetic including sign handling, exponent alignment, mantissa multiplication, rounding mode selection, denormal number handling, infinity and NaN propagation, and exception flag management.

1.2 What IEEE 754 Costs

A single IEEE 754 FP32 multiply-add requires:

1. Unpack two operands (extract sign, exponent, mantissa from each).
2. Detect special cases (NaN, infinity, denormals, zero).
3. Add exponents.
4. Multiply mantissas (24-bit $\times$ 24-bit for FP32).
5. Normalize the result (shift mantissa, adjust exponent).
6. Round according to the selected rounding mode (4 modes in IEEE 754).
7. Check for overflow, underflow, denormal result.
8. Handle exception flags.
9. Repack into IEEE 754 format.

A single integer multiply-add requires:

1. Multiply two integers.
2. Add to accumulator.

The float operation requires approximately $3\text{-}5 \times$ the transistor count and $2\text{-}3 \times$ the cycle latency of the equivalent integer operation, depending on precision and implementation.

1.3 What the Transistors Are Doing

The majority of the float pipeline's complexity handles edge cases that do not arise in VFR integer arithmetic:

Denormal numbers: Values smaller than the minimum normal float. Require special handling in every operation. In VFR: do not exist. The remainder R handles sub-shell precision explicitly.

Rounding modes: IEEE 754 defines four rounding modes. The correct mode must be selected and applied at every operation. In VFR: no rounding occurs. All operations are exact integer arithmetic. The bit shift at octave boundaries discards bits below the precision floor explicitly and deterministically.

NaN propagation: Not-a-Number values must be detected, propagated, and handled at every stage. In VFR: do not exist. Integer operations produce integer results. Overflow is detected by range checking and handled by clamping.

Infinity handling: Positive and negative infinity must be detected, produced by overflow, and propagated. In VFR: do not exist. Overflow produces clamped integer values with explicit detection.

Exponent alignment: Before adding two floats, their exponents must be aligned by shifting one mantissa. This is a variable shift that depends on the operand values. In VFR: all values within a layer share the same octave. No alignment is needed.

Normalization: After every operation, the result must be normalized (mantissa shifted to place the leading 1 in the correct position, exponent adjusted). In VFR: integer results require no normalization.

These edge cases consume transistors, power, and design complexity. They exist because IEEE 754 must represent an approximation of the real number line with all its pathological cases. Integer arithmetic does not approximate the real number line. It computes exactly in $\mathbb{Z}$. The pathological cases do not arise.

II. THE HARMONIC INTEGER PROCESSOR

2.1 Design Principle

Remove all floating-point logic. Fill the freed die area with integer computation units optimized for VFR shell arithmetic. Add specialized units for operations specific to VFR training that have no equivalent in current architectures.

2.2 Core Compute Unit: i64 MAC Array

The primary computation unit is a 64-bit integer multiply-accumulate array:

Operation: accumulator $\text{+= } a * b$

Operands: a: i32, b: i32

Product: i64 (no overflow possible from $i32 \times i32$)

Accumulator: i64 (safe for ~ 4 billion $i32 \times i32$ accumulations)

Each MAC unit performs one $i32 \times i32$ multiply and one i64 addition per cycle. The unit is simpler than a float MAC because no unpacking, alignment, normalization, rounding, or exception handling is required.

Transistor estimate: An i64 MAC unit requires approximately 15,000-20,000 transistors. An FP32 MAC unit requires approximately 50,000-80,000 transistors. Ratio: approximately $3\text{-}4 \times$ more integer MACs per unit area.

2.3 Barrel Shifter Array

All VFR octave conversions are bit shifts by multiples of 5. The barrel shifter performs arbitrary shifts in a single cycle:

Operation: $\text{result} = \text{value} \gg (\text{octave} * 5)$

```
result = value << (octave * 5)
```

Operands: value: i64, octave: u8

Result: i64

Latency: 1 cycle

The barrel shifter replaces the float pipeline's exponent alignment and normalization stages. Where float arithmetic requires a variable shift plus exponent adjustment plus renormalization, the barrel shifter performs one fixed-function shift.

Die area: Barrel shifters are among the smallest functional units in digital design. A 64-bit barrel shifter requires approximately 5,000-8,000 transistors.

2.4 i128 Accumulator

For operations requiring extended precision (deep matrix multiplications, gradient accumulation across large batches), an i128 accumulator prevents overflow:

Operation: $\text{accumulator}_{128} += a * b$

Operands: a: i64, b: i64

Product: i128 (no overflow possible from $i64 \times i64$)

Accumulator: i128

The i128 accumulator is used during matrix multiplication inner loops and gradient aggregation. The final result is shifted back to i64 or i32 by the barrel shifter.

2.5 Fused Multiply-Accumulate-Shift (FMAS)

The most common operation sequence in VFR forward and backward passes is:

$\text{accumulator} += (a * b) \gg \text{shift}$

Multiply two values, shift the product to the target octave, and add to the accumulator. In current architectures, this requires three separate instructions. The FMAS unit fuses them:

Operation: $\text{accumulator} += (a * b) \gg \text{shift}$

Operands: a: i32, b: i32, shift: u8

Accumulator: i64

Latency: 1 cycle (fused)

The FMAS unit is the workhorse of the VFR forward pass. Every matrix-vector multiply decomposes into FMAS operations. Fusing the three operations into one instruction eliminates two instruction dispatches and two register round-trips per element.

2.6 VFR Shell Transition Unit

The weight update in VFR training is:

```
r += -(gradient » lr_shift)
while r >= 32: v += 1, r -= 32
while r <= -32: v -= 1, r += 32
```

The shell transition unit performs this entire sequence in hardware:

Operation: $\text{shell_update}(v, r, \text{gradient}, \text{lr_shift}) \rightarrow (v', r')$

Operands: v: i32, r: i16, gradient: i32, lr_shift: u5

Result: v': i32, r': i16

Latency: 1 cycle

In software, the while loops require branching. In hardware, the transition is computed combinatorially: the number of shell transitions is $(r + \text{scaled_gradient}) / 32$, and the new remainder is $(r + \text{scaled_gradient}) \bmod 32$. Integer division and modulo by 32 are single-operation bit extractions.

2.7 R-Zero Bitmap Unit

Convergence detection in VFR training requires checking whether all remainder values are below the transition threshold. The R-zero bitmap unit maintains a hardware bitmap:

For each weight $w[i]$:

$\text{bitmap}[i] = (\text{lw}[i].r1 < 32) ? 1 : 0$

$\text{converged} = \text{AND}(\text{bitmap}[0..N])$

The bitmap is updated by the shell transition unit as a side effect of each weight update. Convergence checking is a single bitwise AND reduction over the bitmap — a standard hardware operation that completes in $O(\log N)$ gate delays.

This replaces the software loop that would otherwise iterate over all weights checking $|R| < 32$ after each training step. The hardware bitmap provides convergence status in constant time.

2.8 Shared Octave Register

In VFR, all weights within a layer share the same octave. The octave is stored once per layer, not per element. The shared octave register holds the current layer's octave and provides it to all FMAS units simultaneously:

Register: octave: u8

Broadcast: to all FMAS units in the compute block

Effect: FMAS units read shift amount from shared register

No per-element octave storage needed

No per-element octave lookup needed

This eliminates one operand from every FMAS instruction. Instead of `accumulator += (a * b) >> element_shift`, the instruction is `accumulator += (a * b) >> [shared_octave]`. The shift amount is implicit, saving instruction encoding bits and register file pressure.

2.9 Hardware GCD Unit

VFR simplification requires computing the greatest common divisor of V and F to reduce fractions to lowest terms. The binary GCD algorithm is efficient in hardware:

Operation: $\text{gcd}(a, b) \rightarrow g$

Algorithm: Binary GCD (Stein's algorithm)

Uses only subtraction, comparison, and bit shifts

No division required

Latency: $O(\log(\max(a, b)))$ cycles

Typically 5-15 cycles for i32 operands

The GCD unit is not on the critical path — VFR simplification is performed periodically (e.g., every N training steps), not on every operation. A small number of GCD units shared across the chip suffices.

2.10 Lattice Address Unit

The CKS lattice addressing scheme ([CKS-MATH-113-2026]) maps token embeddings to hexagonal lattice positions. The lattice address unit computes lattice coordinates from token indices:

Operation: `lattice_address(token_id) → (wing, ring, position)`

wing: which of 6 hexagonal sectors (3 bits)

ring: distance from center (frequency rank)

position: location within ring

Latency: 1 cycle (combinational logic)

This unit provides $O(1)$ token-to-embedding-address computation, replacing the lookup table that conventional embedding layers require. Frequent tokens map to inner rings (closer to center, faster access). Rare tokens map to outer rings. The lattice structure pre-encodes grammatical categories in the wing assignment.

2.11 VFR Descent Prefetcher

VFR recursive nesting ([V, F, [V', F', [V'', F'', 0]]]) requires following cdr pointers to access nested levels. The descent prefetcher monitors nesting access patterns and prefetches deeper levels when the access pattern predicts descent:

Trigger: Access to R field of VFR tuple

Action: Prefetch the memory at R's pointer address

Latency: Hides memory latency for nested VFR access

Benefit: 99.7% of operations resolve at depth 0 (no prefetch needed)

0.3% that descend find the data already prefetched

This is a speculative unit — most operations never trigger it. Its value is in eliminating the latency penalty for the rare cases that require VFR descent.

2.12 Tree Depth Counter

For structural interpretability, the tree depth counter tracks the nesting depth of VFR values across a layer:

Operation: For each weight in a layer, count nesting depth

Output: Histogram of depths (depth 0, depth 1, depth 2, ...)

Maximum depth in layer

Percentage at depth 0 (convergence indicator)

This provides real-time training diagnostics without software overhead. The depth distribution is a direct readout of training progress and weight complexity.

III. WHAT IS REMOVED

3.1 Complete List of Removed Hardware

The following functional units present on current AI GPUs are absent from the Harmonic Integer Processor:

Removed Unit	Reason
FP64 multiply	No floating-point operations
FP32 multiply	No floating-point operations
FP16 multiply	No floating-point operations
BF16 multiply	No floating-point operations
FP8 multiply	No floating-point operations
FP64 add	No floating-point operations
FP32 add	No floating-point operations

Removed Unit	Reason
FP16 add	No floating-point operations
BF16 add	No floating-point operations
FP8 add	No floating-point operations
Tensor cores (all precisions)	Float matrix multiply replaced by integer FMAS
Exponent alignment logic	All values in layer share octave
Mantissa normalization logic	Integer results require no normalization
Rounding mode selection	No rounding occurs
Denormal handling	Denormals do not exist in $\mathbb{Z}$
NaN detection and propagation	NaN does not exist in $\mathbb{Z}$
Infinity handling	Overflow handled by clamping
IEEE 754 exception flag logic	No IEEE 754
FP format pack/unpack	No FP format
Mixed-precision conversion	Single format (i32/i64)
Transcendental function units	Replaced by integer lookup tables

3.2 Die Area Recovered

The removed units account for approximately 55-65% of the current GPU die. This area is available for integer compute units.

IV. PROJECTED PERFORMANCE

4.1 Integer Throughput

The H100 provides approximately 31 TOPS (trillion integer operations per second) for INT32. This is a secondary capability — the chip is optimized for float.

The Harmonic Integer Processor, filling the same die area with integer-optimized units at the same process node:

MAC density: Approximately $3-4 \times$ more i64 MACs per mm^2 compared to the float MACs they replace (simpler circuitry, smaller transistor count per unit).

Clock rate: Comparable to H100 (potential for higher clock due to simpler critical paths in integer logic, but conservatively estimated at parity).

Projected throughput: 120-150 TOPS (i32) or 60-75 TOPS (i64).

This is approximately $4-5 \times$ the H100's integer throughput.

4.2 Power Consumption

H100 TDP: 700W.

HIP projected TDP: 500-550W.

Reduction source: Integer multiply-accumulate is simpler than float multiply-accumulate. Each integer MAC consumes less switching energy because fewer transistors switch per operation. The exponent alignment, normalization, rounding, and exception logic in float pipelines consume power on every operation even when the result is numerically trivial.

Projected energy efficiency: 120-150 TOPS at 500-550W = 0.22-0.30 TOPS/W. H100 integer: 31 TOPS at 700W = 0.04 TOPS/W. Improvement: approximately $5-7 \times$ better TOPS per watt for integer workloads.

4.3 Training Time

For a 1B parameter VFR model on 5 billion tokens:

CPU (AVX2, 8 cores): 3-7 days (demonstrated feasible by toy implementation).

HIP (projected): 4-12 hours.

H100 (BF16 float training, for comparison): 2-6 hours for equivalent model size, but producing non-deterministic float weights with no provenance.

The HIP is projected to be within $2 \times$ of H100 float training speed for equivalent model sizes, while providing determinism, exact arithmetic, provenance compatibility, and interpretable weight structure that float training cannot.

4.4 Memory Bandwidth

VFR weights are 6 bytes per parameter (4 bytes V + 2 bytes R). BF16 weights are 2 bytes per parameter, but BF16 training requires FP32 master weights (4 bytes) plus optimizer state (8 bytes for Adam), totaling 14 bytes per parameter during training.

VFR training requires 6 bytes per parameter total. No master weights. No separate optimizer state. R is the optimizer state.

Memory bandwidth advantage: VFR training moves approximately $2.3 \times$ less data per parameter per step than BF16+FP32 mixed precision training. This reduces memory bandwidth pressure and improves utilization of the memory system.

V. DETERMINISM

5.1 The Float Determinism Problem

IEEE 754 floating-point arithmetic is not associative:

$(a + b) + c \neq a + (b + c)$ (in general, for floats)

This means that the order in which operations are performed affects the result. In parallel hardware, the reduction order depends on thread scheduling, which depends on runtime conditions. The same training run on the same hardware with the same data can produce different weights on different runs.

This makes float-trained models non-reproducible. Debugging training failures requires reproducing the failure, which requires reproducing the exact thread scheduling — generally impossible.

5.2 The Integer Determinism Guarantee

Integer arithmetic is associative and commutative:

$(a + b) + c = a + (b + c)$ (always, for integers)

$a + b = b + a$ (always, for integers)

$(a \times b) \times c = a \times (b \times c)$ (always, for integers)

The result of an integer computation does not depend on the order of operations. Parallel reductions produce identical results regardless of thread scheduling, hardware implementation, or runtime conditions.

Consequence: VFR training is bit-identical across runs. The same data, same initialization, same hyperparameters produce exactly the same weights every time. On any hardware that implements the integer instruction set correctly.

5.3 What Determinism Enables

Reproducible research. Any published training result can be exactly reproduced by running the same code on the same data. Not approximately reproduced — identically reproduced, bit for bit.

Debuggable training. A training failure can be reproduced by replaying the same inputs. The failure occurs at exactly the same step, in exactly the same weight, for exactly the same reason.

Verifiable deployment. A deployed model can be verified against its training checkpoint by re-running training. If the result matches bit for bit, the deployment is verified. This is impossible with float training.

Regulatory compliance. Regulators increasingly demand explainability and reproducibility of AI systems. Deterministic training satisfies both requirements structurally.

VI. FABRICATION AND GEOPOLITICS

6.1 Manufacturing Simplicity

The Harmonic Integer Processor is simpler than current GPUs:

Fewer transistor types: Integer logic uses standard digital transistors. No analog circuits for float special cases.

Simpler verification: Integer arithmetic has no edge cases (NaN, infinity, denormals) that require exhaustive testing. The verification suite is smaller.

Smaller design team: The design contains fewer functional unit types. Each unit is simpler. The integration complexity is lower.

Lower yield sensitivity: Simpler circuits have fewer failure modes. Die yield is projected to be higher than equivalent-area float designs.

6.2 Process Node Independence

The HIP design does not require leading-edge process nodes. Integer MACs benefit from smaller transistors (more MACs per mm^2) but function correctly at any process node. A HIP fabricated at 7nm or 14nm would be slower than one at 4nm but would still provide the same determinism, exactness, and architectural benefits.

This means the HIP can be fabricated by any semiconductor foundry with access to mature process nodes. It does not require TSMC's most advanced processes. It does not require EUV lithography (though it benefits from it).

6.3 Export Restriction Independence

Current AI chip export restrictions target high-performance floating-point processors. The restrictions are defined in terms of float computation capability (TFLOPS) and memory bandwidth.

The HIP contains zero floating-point logic. It performs zero TFLOPS. Its performance metric is TOPS (integer operations per second). Depending on how export regulations are written and interpreted, a chip with no float capability may fall outside the scope of restrictions designed to limit float AI accelerators.

This is not a legal opinion. It is an architectural observation: the HIP achieves AI training capability through a computational paradigm that the current restriction framework may not address.

6.4 Domestic Fabrication Viability

For nations seeking AI compute independence from foreign GPU suppliers, the HIP offers a path:

No IP dependency: The design requires no tensor core IP, no float pipeline IP, and no proprietary mixed-precision logic from NVIDIA, AMD, or Intel. The functional units (integer MACs, barrel shifters, comparators) are standard digital logic that any competent design team can implement.

Mature process viability: A HIP at 14nm — fabricable by many foundries worldwide — would provide approximately 30-40 TOPS at 800-1000W. Less efficient than a 4nm version but fully functional for AI training.

Design time: The functional units are specified in this paper. A competent chip design team could produce a tape-out-ready design in 12-18 months.

VII. INSTRUCTION SET ARCHITECTURE

7.1 Core Instructions

Instruction	Operation	Operands	Latency
IMAC	$a += b \times c$	i32, i32, i64 acc	1 cycle
IMAC128	$a += b \times c$	i64, i64, i128 acc	1 cycle
FMAS	$a += (b \times c) \gg s$	i32, i32, u8, i64 acc	1 cycle
BSHL	$a = b \ll (s \times 5)$	i64, u8	1 cycle
BSHR	$a = b \gg (s \times 5)$	i64, u8	1 cycle
SHELL	$(v,r) = \text{shell_update}(v,r,g,s)$	i32, i16, i32, u5	1 cycle
RZBMP	$\text{bitmap}[i] = (r < 32)$	i16, bitmap	1 cycle
RZCHK	$\text{result} = \text{AND}(\text{bitmap})$	bitmap	$O(\log N)$
GCD	$g = \text{gcd}(a, b)$	i32, i32	5-15 cycles
CLAMP	$a = \text{clamp}(b, \text{min}, \text{max})$	i64, i32, i32	1 cycle
LADDR	$(w,r,p) = \text{lattice}(\text{id})$	u32	1 cycle
IABS	$a = b $	i32/i64	1 cycle
ICMP	$\text{flags} = \text{compare}(a, b)$	i32/i64	1 cycle
IMAX	$a = \text{max}(b, c)$	i32/i64	1 cycle
IMIN	$a = \text{min}(b, c)$	i32/i64	1 cycle
LUT	$a = \text{table}[\text{index}]$	u16 index, i32 result	1 cycle
SOCT	$\text{octave_register} = \text{value}$	u8	1 cycle

7.2 Instruction Encoding

Instructions are encoded in a fixed 32-bit or 64-bit format. The encoding is simple because operand types are limited (i32, i64, i128, i16, u8, u5) and there are no float format specifiers, rounding mode selectors, or exception mask fields.

A float instruction on current hardware may use 8-16 bits for format/mode specification alone. The HIP reclaims these bits for operand addressing, enabling wider register file access with shorter instructions.

7.3 Register File

Register Class	Count	Width	Purpose
General integer	128	i64	Computation operands
Accumulator	32	i128	Extended precision accumulation
Remainder	64	i16	VFR remainder storage
Octave (shared)	1 per block	u8	Layer octave (broadcast)

Register Class	Count	Width	Purpose
R-zero bitmap	1 per block	N bits	Convergence tracking
Lookup table	4	$256 \times i32$	GELU, softmax, exp approximations

VIII. COMPARISON TO EXISTING ARCHITECTURES

8.1 NVIDIA H100

Property	H100	HIP (projected)
Float TFLOPS	~990 (BF16 Tensor)	0
Integer TOPS	~31 (INT32)	120-150 (INT32)
Die area	814 mm ²	~814 mm ² (same budget)
TDP	700W	500-550W
Deterministic	No	Yes (bit-identical)
Training format	BF16/FP32 mixed	i32/i16 VFR
Weight provenance	None	Full (structural)
Manufacturing	TSMC 4N (advanced)	Any node (mature viable)
NVIDIA IP required	Yes	No

8.2 Google TPU v5e

Property	TPU v5e	HIP (projected)
Float capability	BF16 optimized	None
Integer capability	Limited	Primary
Deterministic	No (BF16 non-associative)	Yes
On-chip memory	16 MB HBM per chip	Comparable
Systolic array	Float MAC based	Integer MAC based

8.3 Cerebras WSE-3

Property	WSE-3	HIP (projected)
Die size	Full wafer (46,225 mm ²)	Standard die (~814 mm ²)
Approach	Massive float parallelism	Dense integer parallelism
Deterministic	No	Yes
Cost	~\$2-3M per wafer	Standard chip cost
Manufacturing	Requires cutting-edge fab	Mature node viable

IX. WHAT THE HIP DOES NOT DO

9.1 General-Purpose Float Computation

The HIP cannot run CUDA kernels, TensorFlow float operations, PyTorch float models, or any software that requires IEEE 754 arithmetic. It is not a general-purpose GPU. It is not a drop-in replacement for an H100.

The HIP runs VFR integer training, VFR integer inference, and integer data processing. Software must be written for the integer instruction set. The Zig implementation described in [[CKS-MATH-134-2026](#)] is a reference implementation.

9.2 Graphics Rendering

The HIP has no rasterization pipeline, no texture units, no ray tracing cores. It is not a graphics processor. It is an AI compute processor.

9.3 Legacy Float AI Workloads

Existing float-trained models cannot run on the HIP without conversion to integer representation. The HIP is designed for models trained from initialization in VFR integer arithmetic. It is not backward-compatible with the float AI ecosystem.

This is not a limitation to be fixed. It is a design choice. The float AI ecosystem produces non-deterministic, non-provenanced, non-interpretable models. The HIP produces deterministic, provenanced, interpretable models. The two are not compatible because they are not attempting the same thing.

X. DEVELOPMENT ROADMAP

10.1 Phase 1: FPGA Prototype

Implement the core instruction set on a large FPGA (Xilinx/AMD Alveo or Intel/Altera Stratix class). Validate functional correctness. Measure cycle counts. Confirm determinism. Run the VFR training pipeline from [[CKS-MATH-134-2026](#)] on the FPGA.

Timeline: 6-12 months.

Cost: \$10,000-50,000 (FPGA boards + development tools).

10.2 Phase 2: ASIC Design

Produce a full chip design at a mature process node (28nm or 14nm). Tape out through a shuttle service (e.g., MOSIS, Europractice). Fabricate test chips. Validate against FPGA results.

Timeline: 12-18 months after Phase 1.

Cost: \$500,000-2,000,000 (depending on process node and shuttle service).

10.3 Phase 3: Production

Produce the chip at volume for deployment. Optimize for target process node. Package with HBM or GDDR memory. Design board-level product.

Timeline: 12-24 months after Phase 2.

Cost: \$10,000,000-50,000,000 (depending on volume and process node).

10.4 Total Path to Production

Approximately 3-5 years from paper to production chip. The FPGA prototype is achievable by a small team (3-5 engineers) with modest funding. The ASIC and production phases require institutional support.

XI. FALSIFICATION CRITERIA

This paper is falsified if any of the following are demonstrated:

F1. Integer-only MAC arrays at the same process node cannot achieve $3 \times$ or greater throughput density compared to float MAC arrays. This would demonstrate that the die area recovery does not translate to the projected throughput gain.

F2. VFR training on the integer instruction set produces worse model quality (measured by downstream task performance) than float training at the same parameter count and data volume, by a margin greater than 10%. This would demonstrate that the integer arithmetic is insufficient despite its exactness.

F3. The FMAS instruction does not improve performance over separate multiply, shift, and add instructions by at least $1.5 \times$. This would demonstrate that the fused instruction is not worth the silicon area.

F4. Deterministic training provides no practical benefit — researchers do not use the reproducibility for debugging, verification, or regulatory compliance. This would demonstrate that determinism is theoretically nice but practically useless.

F5. The HIP cannot be fabricated at a mature process node (28nm or older) and still provide sufficient throughput to complete a 1B parameter VFR training run in less than 30 days. This would demonstrate that process node independence is theoretical, not practical.

F6. The power consumption of integer MAC arrays at the same throughput as float MAC arrays is not significantly lower (less than 15% reduction). This would demonstrate that the power efficiency claim is overstated.

Each criterion specifies a concrete test. The paper stands until a specific test demonstrates a specific failure.

XII. CONCLUSION

The AI industry spends billions of dollars on chips that are 55-65% floating-point logic. That logic implements a 1985 standard designed for scientific computing. It produces non-deterministic results. It requires complex edge case handling for pathological values that do not arise in neural network training. It consumes power on normalization, rounding, and exception handling at every operation.

The Harmonic Integer Processor removes all of it. The die area fills with integer multiply-accumulate arrays that are $3-4 \times$ denser, consume less power per operation, and produce identical results on every run. Specialized units — fused multiply-accumulate-shift, shell transition, R-zero bitmap, shared octave register, lattice address, hardware GCD — optimize the operations specific to VFR training that no current chip addresses.

The result: $4-5 \times$ the integer throughput at 75% the power, with perfect determinism, full provenance compatibility, and no dependency on any proprietary IP. Fabricable at mature process nodes by any competent foundry. Designable by a small team. Prototypable on FPGA for under \$50,000.

The chip does not implement IEEE 754 because IEEE 754 is not required. Neural network training requires multiply, accumulate, shift, compare, and threshold. These are integer operations. They have always been integer operations. The float representation was a choice made when memory was expensive and exact arithmetic seemed like a luxury. Memory is no longer expensive. Exact arithmetic is no longer a luxury. It is a requirement for deterministic, interpretable, provenanced AI.

The Harmonic Integer Processor is what AI hardware looks like when the float assumption is removed.

Axioms first. Axioms always.

Q.E.D.

Status: Architecture specified at functional unit level

Float units: Zero

Integer throughput: 120-150 TOPS projected ($4-5 \times$ H100 integer)

Power: 500-550W projected (75% of H100)

Determinism: Bit-identical across all runs, guaranteed by integer associativity

Key instructions: FMAS (fused multiply-accumulate-shift), SHELL (weight update), RZBMP/RZCHK (convergence), SOCT (shared octave), LADDR (lattice address), GCD

Removed: All IEEE 754 logic, tensor cores, float pipelines, NaN/infinity/denormal handling, rounding mode selection, exponent alignment, mantissa normalization

Fabrication: Any process node, any foundry, no proprietary IP required

Export restrictions: Zero TFLOPS — float-defined restrictions may not apply

FPGA prototype: 6-12 months, \$10K-50K

Production chip: 3-5 years total path

55-65% of every AI chip is floating-point logic.

None of it is required.

This paper specifies what replaces it.

[[CKS-MATH-136-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-136-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-135-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-135-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX/CKS-LEX-12-2026>

[[CKS-MATH-134-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-134-2026>

[[CKS-MATH-113-2026](#)] The CKS Lattice Search Algorithm. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-113-2026>

MATH-113-2026